

Universidad Interamericana de Puerto Rico
Recinto de Arecibo
Programa de Ciencias de Computadoras

Inventario Personal

COMP 2052 — Desarrollo de Aplicaciones Web

Integrantes:

Jadriel Centeno Figueroa (R00667434)

Janiel Valentín Nieves (Y00661894)

Profesor: Javier Dastas Méndez

19 de mayo de 2026

1. Introducción

El presente documento describe el proyecto final **Inventario Personal** desarrollado para el curso COMP 2052. La aplicación permite a los usuarios registrar, organizar y administrar artículos personales mediante una interfaz web. El sistema implementa autenticación con tres roles diferenciados (Admin, Owner y Usuario), operaciones CRUD completas sobre ítems de inventario, y una API REST para pruebas automáticas de los endpoints.

El proyecto se desarrolló utilizando **Flask** como framework de back-end, **MySQL** como base de datos relacional, **SQLAlchemy** como ORM, **Flask-Login** para la gestión de sesiones, **Flask-WTF** para formularios y validaciones, y **Bootstrap 5** con plantillas **Jinja2** para la capa de presentación.

2. Interfaces de la Aplicación

A continuación se presentan las capturas de pantalla que evidencian el funcionamiento completo del proyecto, junto con una descripción de cada interfaz.

2.1 Página de Inicio

La página de bienvenida es la primera vista pública que ve el usuario al ingresar a la aplicación. Muestra el nombre del sistema, una breve descripción de sus funcionalidades principales (Registra, Organiza, Actualiza) y los botones para iniciar sesión o crear una nueva cuenta.



Figura 1. Página de inicio pública de la aplicación.

2.2 Inicio de Sesión

El formulario de inicio de sesión solicita el correo electrónico y la contraseña del usuario. Las credenciales se validan contra la base de datos, y se utiliza *werkzeug.security* para verificar el *hash* de la contraseña. Si las credenciales son válidas, Flask-Login establece una sesión y redirige al usuario al panel principal.

The screenshot shows the 'Iniciar Sesión' (Login) form within the 'Inventario Personal' application. The form is centered on a white background with a light gray border. It features two input fields: 'Email' with the placeholder text 'Ingresa tu email' and 'Contraseña' with the placeholder text 'Ingresa tu contraseña'. Below these fields is a dark gray 'Login' button. At the bottom of the form, there is a link that says '¿No tienes cuenta? [Regístrate aquí](#)'. The application's header is visible at the top, showing the logo 'Inventario Personal' on the left and 'Login Registrarse' on the right.

Figura 2. Formulario de inicio de sesión.

2.3 Registro de Usuario

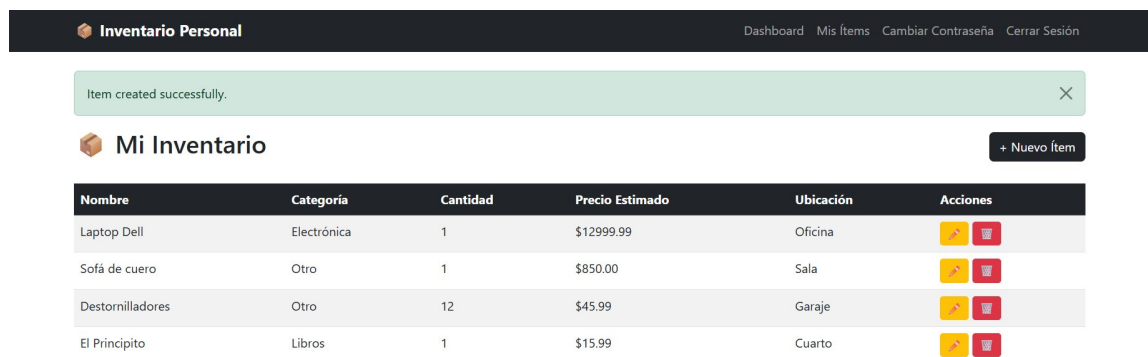
El formulario de registro permite crear una nueva cuenta indicando nombre de usuario, correo, contraseña (mínimo 6 caracteres) y el rol del usuario. Los tres roles disponibles son **Admin**, **Usuario** y **Owner**, cada uno con permisos diferenciados dentro del sistema.

The screenshot shows the 'Crear Cuenta' (Create Account) form within the 'Inventario Personal' application. The form is centered on a white background with a light gray border. It features four input fields: 'Usuario' with the placeholder text 'Tu nombre de usuario', 'Email' with the placeholder text 'Tu email', 'Contraseña' with the placeholder text 'Mínimo 6 caracteres', and 'Rol' with a dropdown menu currently showing 'Admin'. Below these fields is a dark gray 'Registrarse' button. At the bottom of the form, there is a link that says '¿Ya tienes cuenta? [Inicia sesión](#)'. The application's header is visible at the top, showing the logo 'Inventario Personal' on the left and 'Login Registrarse' on the right.

Figura 3. Formulario de creación de cuenta nueva.

2.4 Panel Principal del Owner

Una vez autenticado, el usuario con rol **Owner** accede al panel principal donde se listan únicamente los ítems que él mismo ha creado. Desde esta vista puede consultar el inventario, crear nuevos ítems, editar los existentes o eliminarlos. En la captura se observa además un mensaje de éxito que confirma la creación reciente de un ítem.

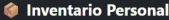


Inventario Personal		Dashboard	Mis Ítems	Cambiar Contraseña	Cerrar Sesión
Item created successfully.					
Mi Inventario		+ Nuevo Ítem			
Nombre	Categoría	Cantidad	Precio Estimado	Ubicación	Acciones
Laptop Dell	Electrónica	1	\$12999.99	Oficina	 
Sofá de cuero	Otro	1	\$850.00	Sala	 
Destornilladores	Otro	12	\$45.99	Garaje	 
El Principito	Libros	1	\$15.99	Cuarto	 

Figura 4. Panel del Owner mostrando sus ítems y mensaje de confirmación.

2.5 Crear Nuevo Ítem

El formulario para crear un nuevo ítem permite ingresar el nombre, categoría (selección entre Electrónica, Muebles, Herramientas, Libros, entre otras), cantidad, precio estimado, ubicación y fecha de adquisición. Los campos opcionales como precio y ubicación están claramente indicados. Al guardar, el ítem se asocia automáticamente al usuario autenticado mediante el campo *owner_id*.

 Inventario Personal

DashboardMis ÍtemsCambiar ContraseñaCerrar Sesión

 Nuevo Ítem

Nombre

Ej: Laptop, Bicicleta...

Categoría

Electrónica

Cantidad

Ej: 1

Precio Estimado (\$)

Ej: 299.99 (opcional)

Ubicación

Ej: Cuarto, Sala... (opcional)

Fecha de Adquisición

mm/dd/yyyy

Guardar

Cancelar

Figura 5. Formulario para registrar un nuevo ítem.

2.6 Editar Ítem Existente

Al seleccionar la opción de edición desde la tabla del panel principal, el sistema carga el mismo formulario pero con los datos actuales del ítem ya pre-poblados. El usuario puede modificar cualquier campo y guardar los cambios. Solo el dueño del ítem (Owner que lo creó) o un Admin tienen permisos para editar.

 Inventario Personal

DashboardMis ÍtemsCambiar ContraseñaCerrar Sesión

 Editar Ítem

Nombre

Laptop Dell

Categoría

Electrónica

Cantidad

1

Precio Estimado (\$)

12999.99

Ubicación

Oficina

Fecha de Adquisición

03/25/2024

Guardar

Cancelar

Figura 6. Formulario de edición con datos pre-poblados.

2.7 Cambiar Contraseña

Cualquier usuario autenticado puede cambiar su contraseña desde esta sección. El sistema requiere la contraseña actual para verificar la identidad antes de actualizar al nuevo valor. La

nueva contraseña se guarda en la base de datos como un *hash* seguro generado con *werkzeug.security*.

Inventario Personal Dashboard Mis Ítems Cambiar Contraseña Cerrar Sesión

Cambiar Contraseña

Contraseña Actual

Nueva Contraseña

Cambiar Contraseña

[← Volver al Dashboard](#)

Figura 7. Formulario de cambio de contraseña.

2.8 Lista de Usuarios (Solo Admin)

Esta vista está restringida exclusivamente al rol **Admin**. Muestra todos los usuarios registrados en el sistema junto con su correo y rol asignado, identificado mediante un *badge* de color: rojo para Admin, amarillo para Owner y azul para Usuario. La opción de menú *Usuarios* solo aparece en la barra de navegación cuando el usuario autenticado tiene rol Admin.

Inventario Personal Dashboard Mis Ítems Usuarios Cambiar Contraseña Cerrar Sesión

Lista de Usuarios Registrados

#	Usuario	Email	Rol
1	Administrator	admin@example.com	Admin
2	Item Owner	owner@example.com	Owner
3	Regular User	user@example.com	Usuario

Figura 8. Lista de usuarios registrados visible únicamente para el Admin.

2.9 Panel Principal del Admin

A diferencia del Owner que solo ve sus propios ítems, el usuario con rol **Admin** visualiza la totalidad de los ítems registrados en el sistema, sin importar quién los haya creado. Esto le permite supervisar todo el inventario y administrar el contenido global de la aplicación.

Inventario Personal

DashboardMis ítemsUsuariosCambiar ContraseñaCerrar Sesión

Mi Inventario

+ Nuevo ítem

Nombre	Categoría	Cantidad	Precio Estimado	Ubicación	Acciones
Laptop Dell	Electrónica	1	\$12999.99	Oficina	 
Sofá de cuero	Otro	1	\$850.00	Sala	 
Destornilladores	Otro	12	\$45.99	Garaje	 
El Principito	Libros	1	\$15.99	Cuarto	 

Figura 9. Panel del Admin con visibilidad sobre todos los ítems del sistema.

3. Pruebas REST del CRUD Principal

Para validar el correcto funcionamiento de los endpoints del CRUD principal se desarrollaron cinco archivos de prueba en formato *.rest*, almacenados en la carpeta **pruebas/** del proyecto. Estos archivos se ejecutan mediante la extensión REST Client de Visual Studio Code. Para activar el modo de pruebas, se modifican las líneas 17-18 del archivo *app/__init__.py* para importar *test_routes* en lugar de *routes*, lo cual sustituye los endpoints con UI por endpoints REST que devuelven respuestas en formato JSON.

3.1 Tabla Resumen de los Endpoints

Archivo	Método	Endpoint	Valores enviados	Valores esperados
create.rest	POST	/items	JSON con nombre, categoria, cantidad, precio_estimado, ubicacion, fecha_adquisicion, owner_id	201 Created + JSON con id y owner_id
read.rest	GET	/items	Ninguno	200 OK + lista JSON de todos los ítems
read-a-row.rest	GET	/items/<id>	ID del ítem en la URL	200 OK + JSON del ítem o 404 Not Found
update.rest	PUT	/items/<id>	ID en URL + JSON con los campos a actualizar	200 OK + JSON con mensaje e id
delete.rest	DELETE	/items/<id>	ID del ítem en la URL	200 OK + JSON con mensaje e id

3.2 Archivo *create.rest* (POST)

Crea un nuevo ítem en la base de datos a partir de los datos enviados en formato JSON. El endpoint recibe los campos del ítem y el *owner_id* del usuario dueño. Si los datos son válidos, retorna el código HTTP 201 (Created) junto con el ID asignado al nuevo ítem.

```
### Crear nuevo item, POST

POST http://localhost:5000/items
Content-Type: application/json

{
  "nombre": "Laptop Dell XPS 13",
  "categoria": "Electronica",
  "cantidad": 1,
  "precio_estimado": 1599.99,
  "ubicacion": "Oficina",
  "fecha_adquisicion": "2025-05-15",
  "owner_id": 2
}
```



```
pruebas > create.rest > POST /items
1  ### Crear nuevo ítem, POST
2
3  Send Request
4  POST http://localhost:5000/items
5  Content-Type: application/json
6  {
7    "nombre": "Laptop Dell XPS 13",
8    "categoria": "Electronica",
9    "cantidad": 1,
10   "precio_estimado": 1599.99,
11   "ubicacion": "Oficina",
12   "fecha_adquisicion": "2025-05-15",
13   "owner_id": 2
14 }

http://1...0/
1  HTTP/1.1 201 CREATED
2  Server: Werkzeug/3.1.8 Python/3.14.3
3  Date: Tue, 19 May 2026 21:00:45 GMT
4  Content-Type: application/json
5  Content-Length: 59
6  Vary: Cookie
7  Connection: close
8
9  {
10   "id": 8,
11   "message": "Item creado",
12   "owner_id": 2
13 }
```

Figura 10. Ejecución de *create.rest*: petición POST y respuesta 201 Created.

3.3 Archivo *read.rest* (GET todos)

Obtiene la lista completa de todos los ítems registrados en la base de datos. No requiere parámetros. La respuesta es un arreglo JSON donde cada elemento contiene los datos completos de un ítem.

```
### Obtener todos los items, GET

GET http://localhost:5000/items
Content-Type: application/json
```

```
pruebas > read.rest > GET /items
1  ## Obetener todos los items, GET
2
3  Send Request
4  GET http://localhost:5000/items
5  Content-Type: application/json

http://1...0/
1  HTTP/1.1 200 OK
2  Server: Werkzeug/3.1.8 Python/3.14.3
3  Date: Tue, 19 May 2026 21:02:53 GMT
4  Content-Type: application/json
5  Content-Length: 1778
6  Vary: Cookie
7  Connection: close
8
9  [
10   {
11     "cantidad": 1,
12     "categoria": "Otro",
13     "fecha_adquisicion": "2023-08-20",
14     "id": 2,
15     "nombre": "Sof\u00e1 de cuero ",
16     "owner_id": 2,
17     "precio_estimado": 850.0,
18     "ubicacion": "Sala"
19   },
20   {
21     "cantidad": 12,
22     "categoria": "Otro",
23     "fecha_adquisicion": "2025-01-10",
24     "id": 3,
25     "nombre": "Destornilladores",
26     "owner_id": 2,
27     "precio_estimado": 45.99,
28     "ubicacion": "Garaje "
29   },
30   {
31     "cantidad": 1,
32     "categoria": "Libros",
33     "fecha_adquisicion": "2025-12-05",
```

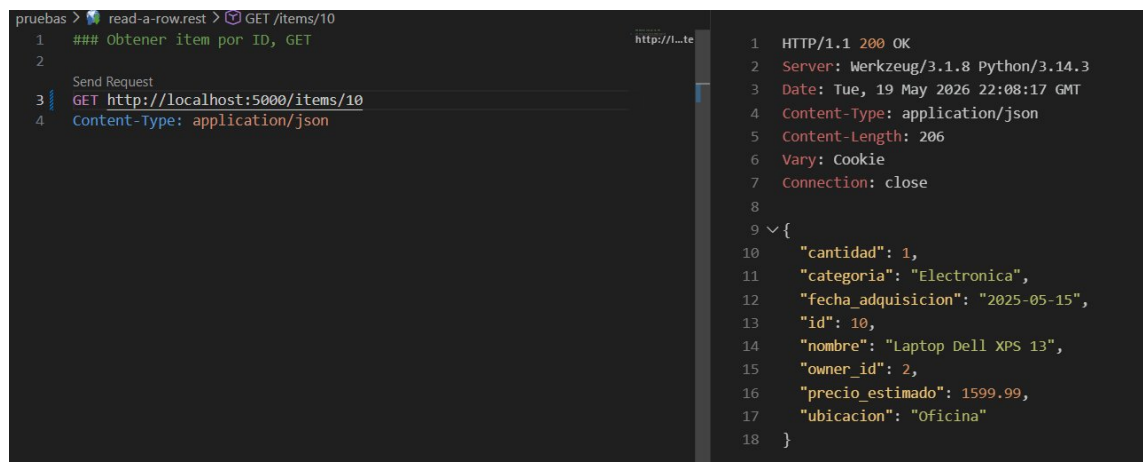
Figura 11. Ejecución de *read.rest*: respuesta 200 OK con el arreglo de ítems.

3.4 Archivo *read-a-row.rest* (GET por ID)

Obtiene los datos de un ítem específico utilizando su ID en la URL. Si el ítem existe, retorna 200 OK con el JSON del ítem. Si no existe, retorna 404 Not Found.

```
### Obtener item por ID, GET
```

```
GET http://localhost:5000/items/10
Content-Type: application/json
```



```
pruebas > read-a-row.rest > GET /items/10
1  ### Obtener item por ID, GET
2
3  Send Request
4  { GET http://localhost:5000/items/10
   Content-Type: application/json

1  HTTP/1.1 200 OK
2  Server: Werkzeug/3.1.8 Python/3.14.3
3  Date: Tue, 19 May 2026 22:08:17 GMT
4  Content-Type: application/json
5  Content-Length: 206
6  Vary: Cookie
7  Connection: close
8
9  {
10  "cantidad": 1,
11  "categoria": "Electronica",
12  "fecha_adquisicion": "2025-05-15",
13  "id": 10,
14  "nombre": "Laptop Dell XPS 13",
15  "owner_id": 2,
16  "precio_estimado": 1599.99,
17  "ubicacion": "Oficina"
18  }
```

Figura 12. Ejecución de *read-a-row.rest*: respuesta 200 OK con un ítem específico.

3.5 Archivo *update.rest* (PUT)

Actualiza un ítem existente. Recibe el ID en la URL y los campos a modificar en el cuerpo JSON. Solo se actualizan los campos enviados, manteniendo los demás con sus valores actuales. Retorna 200 OK con un mensaje de confirmación.

```
### Editar un item, PUT
```

```
#Simulando PUT con nuevos datos del item
```

```
PUT http://localhost:5000/items/10
Content-Type: application/json
```

```
{
  "nombre": "Laptop Dell XPS 13 (actualizada)",
  "cantidad": 2,
  "precio_estimado": 1599.99,
  "ubicacion": "Sala"
}
```

```
pruebas > update.rest > PUT /items/10
1  ### Editar un item, PUT
2
3  #Simulando PUT con nuevos datos del item
4
5  Send Request
6  PUT http://localhost:5000/items/10
7  Content-Type: application/json
8
9  {
10     "nombre": "Laptop Dell XPS 13 (actualizada)",
11     "cantidad": 2,
12     "precio_estimado": 1599.99,
13     "ubicacion": "Sala"
14 }

1  HTTP/1.1 200 OK
2  Server: Werkzeug/3.1.8 Python/3.14.3
3  Date: Tue, 19 May 2026 22:09:18 GMT
4  Content-Type: application/json
5  Content-Length: 48
6  Vary: Cookie
7  Connection: close
8
9  {
10     "id": 10,
11     "message": "Item actualizado"
12 }
```

Figura 13. Ejecución de `update.rest`: respuesta 200 OK confirmando la actualización.

3.6 Archivo `delete.rest` (DELETE)

Elimina permanentemente un ítem de la base de datos utilizando su ID. Retorna 200 OK junto con un mensaje confirmando la eliminación. Si el ID no existe, retorna 404 Not Found.

```
### Eliminar item, DELETE

DELETE http://localhost:5000/items/8
```

```
pruebas > delete.rest > DELETE /items/8
1  ### Eliminar item, DELETE
2
3  Send Request
4  DELETE http://localhost:5000/items/8
5
6
7
8
9  {
10     "id": 8,
11     "message": "Item eliminado"
12 }
```

Figura 14. Ejecución de `delete.rest`: respuesta 200 OK confirmando la eliminación.

4. Repositorios en GitHub

Cada integrante mantiene una copia completa del proyecto en su propio repositorio público de GitHub, incluyendo el código fuente, el archivo README de documentación y este documento PDF.

Integrante	Dirección del repositorio
Jadriel Centeno Figueroa	https://github.com/FPSHK/inventarioPersonalJadriel
Janiel Valentín Nieves	https://github.com/ALLTRIU/ProyFinalcomp2052

5. Conclusión

El proyecto Inventario Personal cumple con los requisitos establecidos en el curso: implementa autenticación con manejo de sesiones, control de acceso basado en roles, operaciones CRUD completas sobre la entidad principal del sistema, y endpoints REST documentados y probados. La estructura modular del código, el uso de un ORM y la separación entre lógica de back-end y presentación permiten que el proyecto sea mantenible y extensible.